Facultatea de Automatică şi Calculatoare, UPB
Departamentul de Calculatoare

Analiza algoritmilor

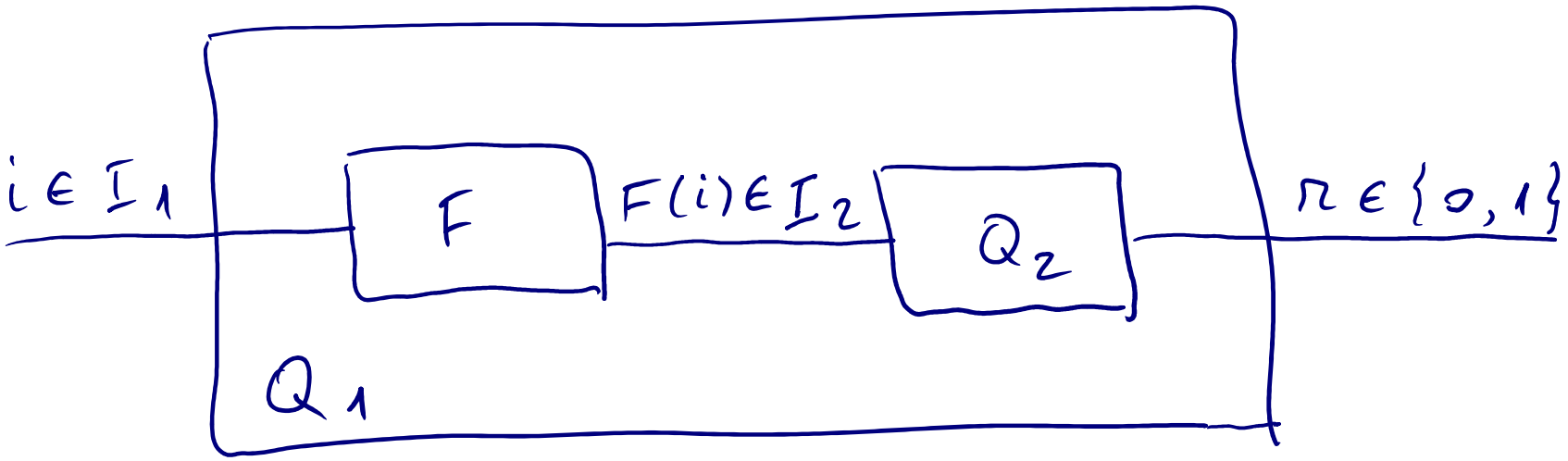
ŞL.dr.ing. Andrei Mogoş

2020-2021

- Clase de probleme (2) -

Def: Fie $Q_1, Q_2 \in Dec$. Spunem că problema Q_1 este reductibilă în timp polinomial la problema Q_2, relație notată $Q_1 \leq_p Q_2$, dacă există un algoritm determinist tractabil F care transformă problema Q_1 în problema Q_2.

- F transformă datele $i \in I_1$ ale problemei Q_1 în date $F(i) \in I_2$ ale problemei Q_2 astfel încât: $\forall i \in I_1$ avem $Q_1(i) = 1 \iff Q_2(F(i)) = 1$.



$$Q_1 \leq_p Q_2$$

(Q_1 este reductibilă în timp polinomial la Q_2)

Prop 1: Dacă $Q_2 \in P$ și $Q_1 \leq_p Q_2$ atunci $Q_1 \in P$.

Prop 2: Relația $\leq_p$ este tranzitivă:

$\forall Q_1, Q_2, Q_3 \in Dec$, dacă $Q_1 \leq_p Q_2$ și $Q_2 \leq_p Q_3$ atunci $Q_1 \leq_p Q_3$.

Obs: Prop 1 și Prop 2 se demonstrează folosind definiția relației $\leq_p$.

Def: O problemă $Q \in Dec$ s.n. **NP-dură** dacă $\forall Q' \in NP$ avem $Q' \leq_p Q$.

- NPD = mulțimea problemelor NP-dure

Def: O problemă $Q \in Dec$ s.n. **NP-completă** dacă Q este NP-dură și $Q \in NP$.

- NPC = mulțimea problemelor NP-complete

Obs: Problemele NP-complete sunt cele mai dificile probleme din clasa NP.

Obs: Problemele din clasa P sunt cele mai simple probleme din clasa NP.

(T1) $Q_2 \in NPD \Leftrightarrow \exists Q_1 \in NPC$ astfel încât $Q_1 \leq_p Q_2$.

(T2) $\forall Q_1, Q_2 \in NPC$ avem $Q_1 \leq_p Q_2$ și $Q_2 \leq_p Q_1$.
(Problemele din NPC sunt echivalente în raport cu relația $\leq_p$)

(T3) Dacă $NPC \cap P \neq \emptyset$ atunci $P = NP$.

(C1) Dacă $NPC \cap P \neq \emptyset$ atunci $NPC \subseteq P$.

(T4) Dacă $NPD \cap P \neq \emptyset$ atunci $P = NP$.

(C2) Dacă $NPD \cap P \neq \emptyset$ atunci $NPC \subseteq P$.

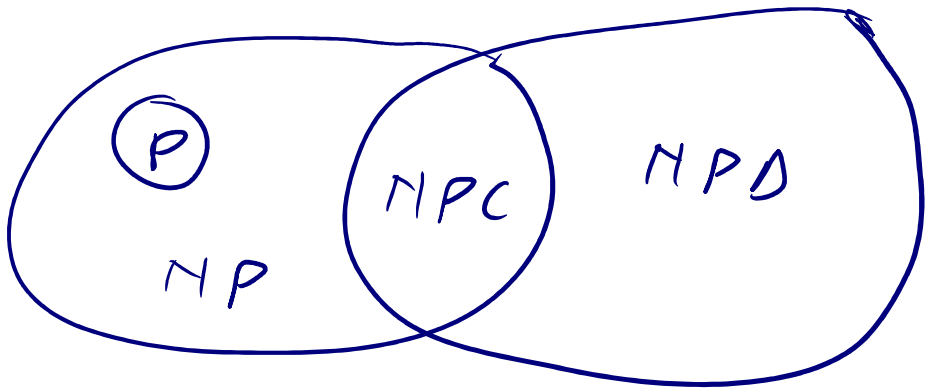
Obs: Teoremele 3 și 4 spun că dacă o anumită proprietate este adevărată atunci $P = NP$.

- Problema "$P = NP$?" este cea mai importantă problemă din informatica teoretică.

- Structură posibilă a spațiului problemelor:

Știm că:

- $P \subseteq NP$
- $NPC = NPD \cap NP$



Teorema Cook

Def: S.n. **formulă propozițională în formă normală conjunctivă** (CNF) o formulă de forma:

- variabilă = simbol care poate fi asociat unei unice valori din {0, 1}
- literal = variabilă | ¬ variabilă
- termen = literal [∨ literal]*
- formulă = termen [∧ termen]*

$\neg$ = not, $\vee$ = sau logic, $\wedge$ = și logic

Def: O formulă CNF F s.n. **satisfiabilă** dacă există o asociere a variabilelor formulei F la valori din $\{0,1\}$ astfel încât valoarea lui F să fie 1.

Ex: $F = (x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3)$

$x_1 = 1$
$x_2 = 0$
$x_3 = 0$

$\Rightarrow F(1,0,0) = (1 \vee 0 \vee 0) \wedge (0 \vee 1 \vee 0) =$

$= 1 \wedge 1 = 1$

F este CNF și F este satisfiabilă

Def: S.n. **SAT** problema:

"Fiind dată o formulă CNF F să se stabilească dacă F este satisfiabilă!" Dacă n este numărul variabilelor din formulă considerăm că numărul termenilor formulei este $m = O(n^k)$.

Obs: Problema SAT mai este numită și CNF - SAT.

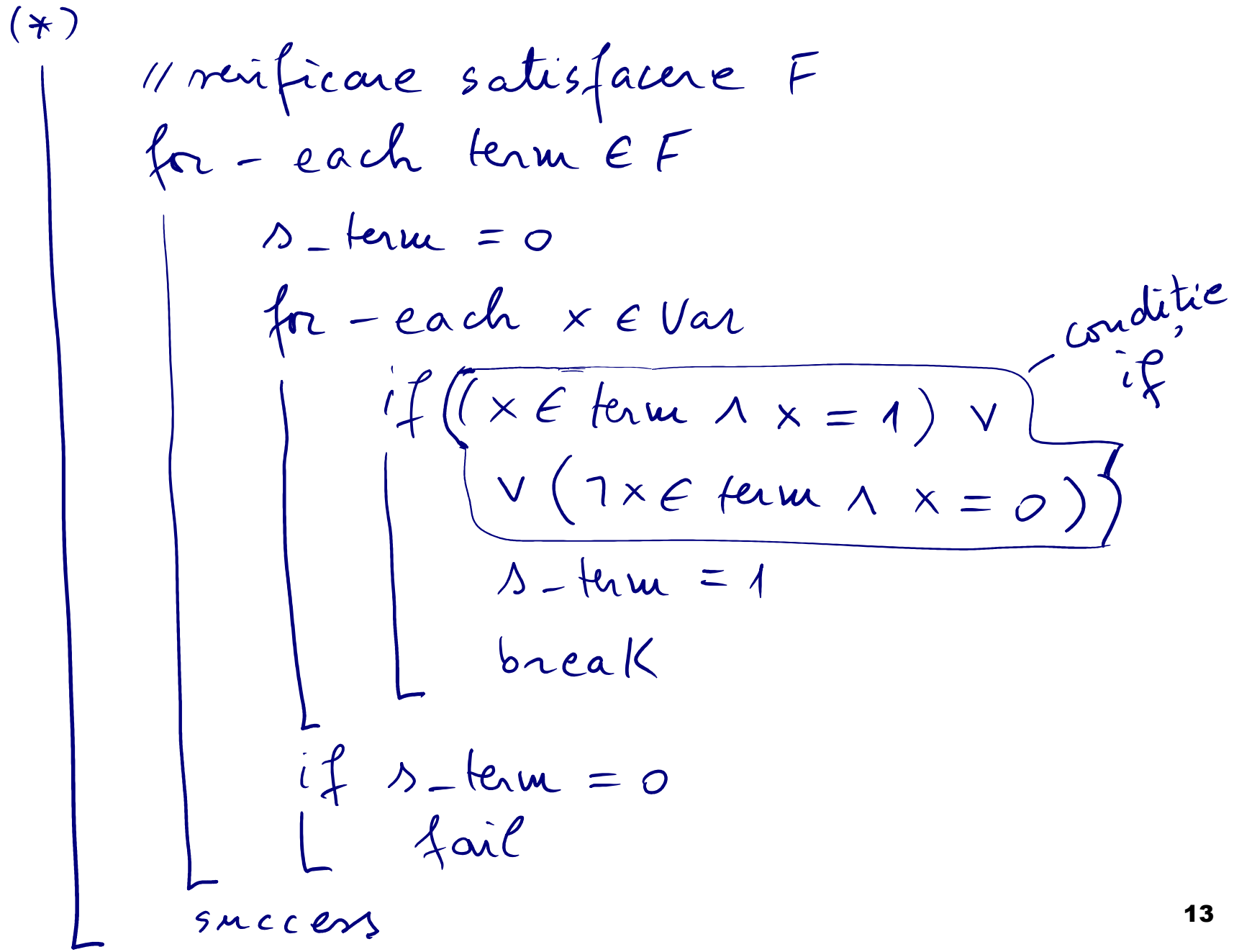
(T) Teorema Cook: $SAT \in P \iff P = NP$.

Dem

$P = NP \Rightarrow SAT \in P$

- Construim un algoritm nedeterminist tractabil care să rezolve problema SAT.

```
M_SAT(F)
    Var = variabile(F)
    for-each x ∈ Var
        x = choice({0,1})
(*)
```


(*)
// verificare satisfacere F
for - each term ∈ F
s_term = 0
for - each x ∈ Var
if ((x ∈ term ∧ x = 1) ∨
∨ (¬x ∈ term ∧ x = 0))
conditie if
s_term = 1
break
if s_term = 0
fail
succes

- Fie n numărul variabilelor formulei, iar $m = O(n^k)$ este numărul termenilor din formulă.
- Complexitatea algoritmului N_SAT este:

$$O(n^k \cdot n \cdot n^c) = O(n^{k+c+1})$$

n^k — număr termeni; n — număr variabile; n^c — complexitatea căutării unui literal într-un termen

=> N_Alg este algoritm nedeterminist tractabil care rezolvă problema SAT.

$\Rightarrow SAT \in NP$

Conform ipotezei: $P = NP$ $\Big| \Rightarrow SAT \in P$.

- Explicație legată de algoritm:

```
for-each x ∈ Var
    x = choice({0,1})
    ⋮
```

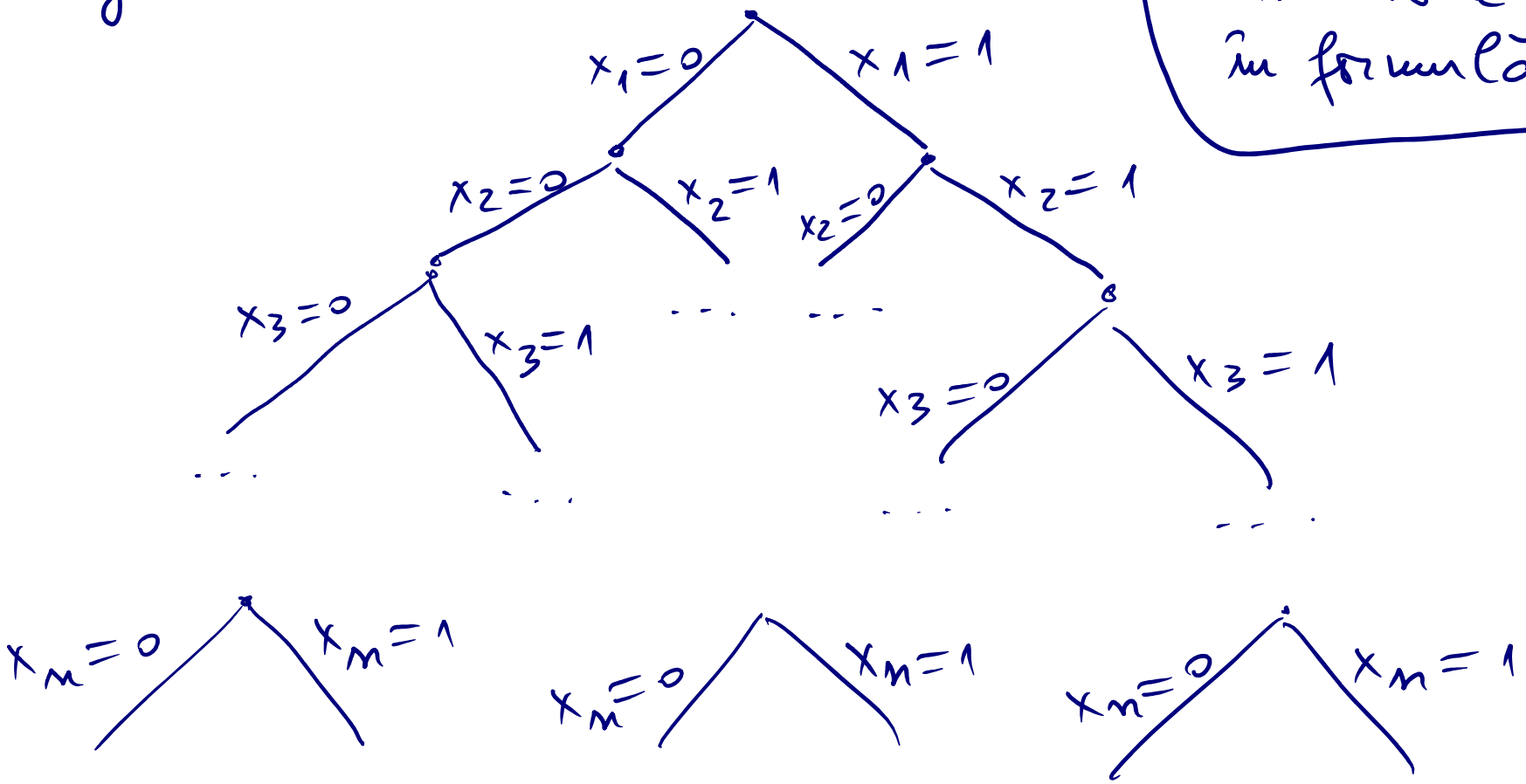
are următorul efect:

- pentru fiecare variabilă x generează o copie a algoritmului cu $x = 0$ și o copie a algoritmului cu $x = 1$;

- amele copii execută din nou corpul instrucțiunii for-each și generează alte două copii ale algoritmului pentru următoarea variabilă în același mod în care au fost generate și pentru prima variabilă;
- se continuă în același mod până la utilizarea tuturor variabilelor formulei.

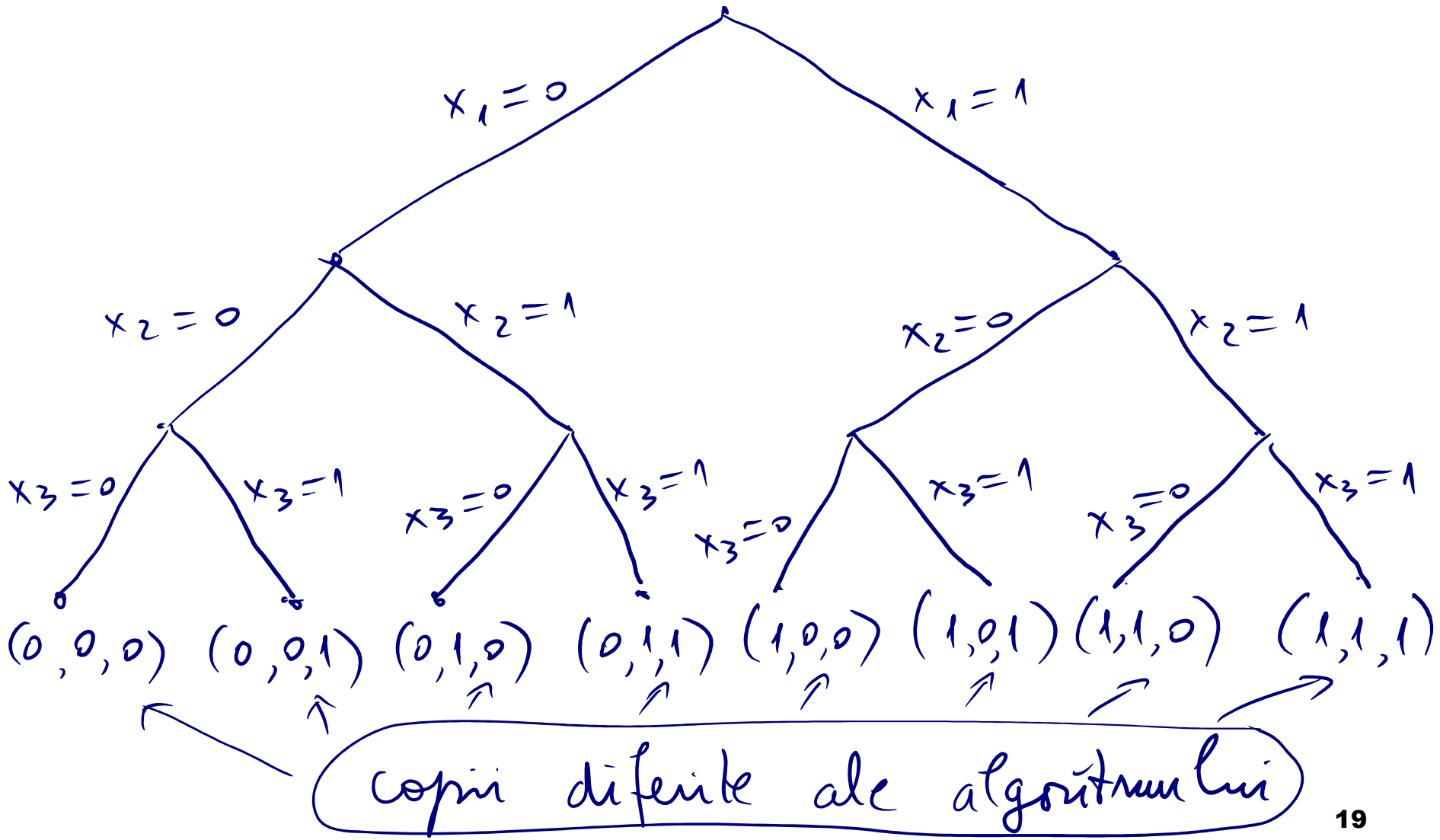
Rezultă următorul arbore de căutare ale algoritmului:

n variabile în formulă



- Fiecare cale de la rădăcină la o frunză reprezintă o cale (o copie a algoritmului inițial);
- Fiecare astfel de copie a algoritmului face o asociere a variabilelor formulei la valori din $\{0, 1\}$;
- Fiecare astfel de copie a algoritmului va executa toate operațiile din algoritm de după for-ul analizat.

Pentru $n = 3$ avem:



- Fiecare copie verifică dacă pentru asocierea respectivă a variabilelor formulei la valori din {0,1} valoarea formulei este 1:
  - dacă $F(x_1, x_2, \ldots x_n) = 1$ atunci copia respectivă a algoritmului va executa operația <u>success</u> care va termina întregul algoritm cu succes (și celelalte copii vor fi terminate);

- dacă $F(x_1, x_2, \ldots, x_n) = 0$ atunci copia respectivă a algoritmului va executa operația fail care va termina cu insucces această copie a algoritmului (celelalte copii ale algoritmului nu sunt afectate).

Prin urmare, este suficient ca o singură copie a algoritmului să găsească $F(x_1, x_2, \ldots, x_n) = 1$ pentru ca algoritmul să întoarcă valoarea 1.

$SAT \in P \Rightarrow P = NP$

- Se arată că problema SAT este NP-dură folosind definiția unei probleme NP-dure:

  $\forall Q' \in NP$ avem $Q' \leq_p SAT$

- Demonstrația faptului că SAT este NP-dură nu se prezintă în cadrul cursului. Cine este interesat poate consulta demonstrația din cartea domnului profesor Ciurea, carte indicată în bibliografia cursului.

(1) SAT este NP-dură

(2) SAT $\in$ NP (am arătat acest lucru în cadrul demonstrației pentru cealaltă implicație independent de ipoteza acestei implicații)

Din (1) și (2) $\Rightarrow$ SAT este NP-completă (SAT $\in$ NPC)

SAT $\in$ NPC

Din ipoteză: SAT $\in$ P $\Big|$ $\overset{T3}{\Rightarrow}$ P = NP

Obs:

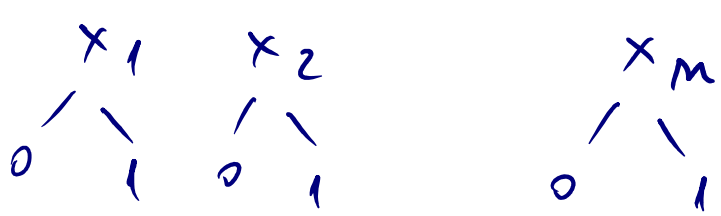
- $SAT \in NPC$ (deoarece s-a demonstrat acest lucru în cadrul demonstrației pentru Teorema Cook fără a folosi ipotezele celor două implicații);
- Din obs. precedentă rezultă $NPC \neq \emptyset$;
- SAT este o problemă foarte importantă în informatica teoretică;
- în cursul nostru SAT va fi rădăcina arborelui de probleme NP-complete.

Rezolvarea problemei SAT folosind un algoritm determinist - rezolvarea clasică

- se generează toate asocierile variabilelor formulei la valori din $\{0, 1\}$
- pentru fiecare asociere de acest fel se verifică dacă valoarea formulei este 1.

Obs: Dacă pentru cel puțin o astfel de asociere valoarea formulei este 1 atunci algoritmul întoarce valoarea 1.

$2 \cdot 2 \cdot \ldots \cdot 2 = 2^n$ asocieri ale variabilelor formulei la valori din $\{0,1\}$ (n variabile)



- Deoarece algoritmul este <u>determinist</u> rezultă că atât generarea asocierilor, cât și verificarea valorii formulei pentru fiecare asociere se fac <u>serial</u>.
- 2^n asocieri pentru care formula este calculată $\Rightarrow$ complexitatea acestui algoritm este $\Omega(2^n)$. → cazul cel mai defavorabil